

Interpolating Scanner

Front Panel

If someone would ask me about my favorite synthesizer module, I would not hesitate to name this one. It is quite simple in its structure, and I found a quite simple way to implement it, too (but not before trying and discarding much more complicated circuits) - so what's it all about?

Two Ideas Combined ...

... A Voltage Controlled Mixer (1)

Every modular synthesizer should have voltage controlled mixers in one form or the other, to create a dynamical mix of several sound sources, several waveforms, several filter outputs and so on. The basic form is to patch several ordinary VCAs into an ordinary signal mixer. Some systems already have these modules combined as a compact Voltage Controlled Mixer module, with a number of Signal inputs and the same number of CV inputs to control the level of each input channel. So each signal source can have its own envelope, for example, and it is possible to create a dynamic mix from several partial sounds. This approach surely has its benefits, and can be the basis for an additive synthesis.

Sometimes, however, you actually do not *want* that much individual control. Sometimes a *separate* envelope or CV for each signal is just too much to patch and to take care about. Sometimes you want to change the sound with one or two CVs and keep the overall volume constant (and use a separate envelope for your master volume / master VCA). That is why other control schemes for VCAs were invented.

A Voltage Controlled Crossfade module, for example, consists of two VCAs with their outputs summed together, and with their gain factors controlled inversely by one single CV.

Another, more advanced example is the so called *Vector Synthesis*, as it is implemented in the SCI Prophet VS. Here we have 4 input channels that are controlled by 2 CV's, represented by the (x,y)-position of a point in a certain area. This is a very clever way to control the relative levels of 4 sources with only 2 CVs, because many different combinations of the sources are available by moving a point around this area. If you look very careful, however, you will note that there are some constraints about possible combinations and about the transitions between certain combinations, because of the fact that all 4 sources share one planar area. That's the price to pay for easy access to 4 sources with just 2 CV's. (Think of it, a full 4-channel VC Mixer would not work with an X-Y Joystick in a plane, but with 4 orthogonal vectors in a 4-dimensional hyperspace (;->))

In some way, the Interpolating Scanner uses even less control for even more input channels. It performs a *linear* crossfade over N input channels (in my module, N=8 inputs) with only one

Control Voltage. (We refer to this CV input as "Scan Input", or "Scan CV", to avoid mixing it up with the input channels. This is important, because we will see later that the Signal Inputs and Scan inputs can change their functions in various ways.) [[Block Diagram](#)]

It works like this: At a certain voltage (say, 1V) on the Scan Input, Input Channel 1 is selected. That means, the VCA for Signal Input 1 is fully on, and all the others are fully off. As we increase the Scan CV slowly from 1V to 2V, VCA1 will be closed more and more, and VCA2 will be opened accordingly. So we perform a crossfade from Input 1 to Input 2. When the Scan CV is at 2V, VCA2 is the only one to conduct. If we further increase the Scan CV, VCA2 gain will decrease and VCA3 will start to conduct, and so on. At 8V, the last channel is selected. In other words, we do a crossfade "along the line", over all Inputs one by one. For those who know the interiors of old Hammond Organs, the famous "Scanner Vibrato" works just like this (only that it's a mechanical device, and it's circular, i.e. start point and end point are the same). That's where I borrowed the name for my module. And, though I haven't done it yet, emulating a Hammond Vibrato might be one interesting application !

The momentary scanning position is monitored by 8 LEDs, one beneath each Signal Input jack. Just like the VCAs, the LEDs are not abruptly switched on and off, but they go bright and dimm to reflect the gain of the corresponding VCA.

So far, we have been speaking of *one* Scan CV, but of course we can mix several modulation sources like envelopes and LFOs together for the Scan operation. In fact, my module has one "Manual Scan" Knob to set the static bias point along the line, and two CV inputs "Scan Left" and "Scan Right" with modulation depth knobs to match. But the general idea (and constraint) is that we have only *one* parameter for control, and only a mix of 2 adjacent Signal Inputs at one time. What looks like a heavy constraint at first, is of high practical use instead, as the following examples may show:

One of the first experiments I did was patching the outputs of an EMS 8 Octave Filter Bank into the 8 scanner inputs. Scanning along the line in their regular order is quite impressive already. Then change the order of the filter outputs (like the cross patching that is possible on some vocoders) for more variation.

Or take a State Variable Filter (like the 12dB/Oct filter of the Oberheim SEM) and patch the HP, LP, and BP outputs into 3 Scanner Inputs. Ever dreamed of the SEM's filter mode knob to be replaced by a CV ? Here it is, and you don't even need an extra click stop position for BP. The Notch Filter function will be produced at a 50% mix between LP and HP. Or do you want the BP in between LP and HP? Just patch it, and experiment. (In my JH-4 Polysynth, I have implemented a small, 4-channel Scanner that fades between LP, HP, BP of a 12dB-Filter, and LP of a 24dB filter as 4th position.)

... And A Piecewise Linear Voltage Controlled Waveshaper (2)

There's pretty much that can be done with the classic synthesizer waveforms saw, triangle and pulse, together with analogue filters. The other way, starting with a waveform of low harmonic content, and running it thru certain waveshaper devices, leads to very interesting results as well. The idea is

to have a nonlinear gain stage, with either the level of the input signal variable, or the function of nonlinearity changed by some CV, in order to get a varying spectrum from a static input waveform.

The most common waveshaping devices are (guitar) distortion pedals. For analogue synthesizers, there are various waveshaper modules or "timbre modulators" from different manufacturers, cleverly designed for a special change of waveform that is useful for musical applications. Like various filters, these various waveshapers all have a certain "character" of their own, and the specific circuits of some modules have been subject to speculation amongst diy builders.

Another way to get uncommon waveforms is using one or the other form of a weighted pulse technique. Some analogue sequencers can be clocked at audio rate, and the individual steps can be used to create a staircase type audio waveform. A similar approach (that doesn't involve frequency division) is driving a set of comparators with different thresholds (like a flash ADC), and weighting the comparator outputs with potentiometers to create a new waveform. A third variety of that type of waveforms is the mixing of square waves at different footage (like the Roland SH-7 or Korg Poly-800 does). Because of the partial elements being square (and not sine) waves, this is not Fourier synthesis; just another means to get interesting staircase-type waveforms.

Unfortunately, all these staircase-type waveforms also share a certain type of spectral content. To call them "hollow" would be too derogatory, but they certainly sound like "composed of square waves", which they actually are. It's a matter of personal taste, of course, but I believe that things become better when the transitions are somewhat smoothed. You can use a tracking filter for that, or start with a continuous waveshaping technique in the first place.

The technical term for that is "Piecewise Linear" (PWL), and that's what the second idea of the Interpolating Scanner is about. Piecewise linear approximation of a certain curve is like redrawing the curve with a polygon. The curve is not completely smooth yet. It still has corners, but it doesn't have steps anymore. So we still don't have a smooth "analogue" curve, but we're closer. And you can hear this improvement. (Note: The next step would be an approximation with cubic splines. It was fun to implement that for circuit simulators, see US patent # 5594851 and #5612907, if you are interested. Anybody volunteering to implement that in analogue hardware ? (;->)

So the Interpolating Scanner can work as Piecewise Linear Waveshaper. You set up 8 "breakpoints", and a nonlinear curve will be created with straight line segments that connect these breakpoints. If you have an Oberheim Matrix synthesizer, you're already familiar with this. Implementing it in analogue hardware has some advantages, however. To use the Interpolating Scanner for PWL waveshaping, simply do the scanning at audio rate (plug a VCO's triangle output into the "Scan Left" input, for example), and use different DC voltages for the 8 input channels. To achieve this, don't plug anything into the 8 input jacks, and switch on the "DC bias" switch. Then each channel gets a fixed DC voltage which can be attenuated with the individual channel input level knobs. (See [Block Diagram](#))

The first thing you can do with the IS as PWL Waveshaper is creating a static nonlinear curve. It does not have to be monotonous, so frequency multiplying is possible as well. Then drive it with varying input amplitude as if it were a distortion pedal. Play single notes (single oscillators) and you get an altered waveform. Play fifths and octaves (several oscillators), and you get the familiar distortion and intermodulation beating, just as with other distortion devices. Play less

harmonic intervalls or chords and you're in distortion hell. You have a lot of freedom to change the character by changing the breakpoint levels. (It won't become as smooth as a tube or FET overdrive, though.) Input signals of low harmonic content like triangle or sine are preferable to drive the Scanner.

The next thing to try is applying a variable offset voltage. For a first test, build a single peak in the curve, by setting one breakpoint to maximum and the other ones to zero. You will get something like a pulse waveform from a triangle input, only that the pulse is more like a triangle or trapezoid. Then simply add an audio (triangle) signal and a slowly changing DC voltage (LFO, ENV, ...) to drive the Scanner. You will get results that sound similar to pulse width modulation or pulse position modulation. Then create two or more peaks in the polygon. Try (0, 0, 1, 0, 1, 0, 0, 0) for the breakpoint potentiometers. Or create a region with a single pulse, plus a region with a double pulse, like (1, 0.5, 0, 0.5, 0.5, 0.5, 1, 0.5), and set the triangle input level so that it only comprises one region at a time. Then use an LFO for modulation inside this local regions, and an Envelope signal of greater amplitude to shift the bias point from the single pulse region to the double pulse region dynamically.

Now it's time to change the breakpoints themselves with control voltages. Remember that each potentiometer is connected to a signal input; it's just normalized to a fixed voltage if you don't use the input jack, and turn on the Offset switch. But you can replace the fixed voltage of each input channel by any dynamically changing voltage. Then your momentary breakpoint at this certain position will be changed with that external CV. You can connect 8 Envelope Generators here if you like, for complete dynamic control of the whole PWL curve. But in practice it's more than enough to have a majority of static breakpoints and only one or two modulated. Think of it: One single CV (from a Pedal, an Aftertouch, Velocity, ...) is enough to dynamically "raise" an additional triangle-shaped peak in your previous curve, which causes the spectrum to change completely.

Such use of modulation signals is not limited to slowly changing voltages. As the CV inputs are intended to work as audio mixer inputs in the "Scanner" mode, they have full audio bandwidth. Replace one static breakpoint by a second audio signal, and you will get ring-modulation like output signals, whenever the amplitude + DCshift of your scanning waveform comprises the region of the modulated breakpoint.

So far a few basic applications for a start. There are a lot of different possibilities to patch this module, several synthesis techniques combined, and it is no problem to create the weirdest sounds with just 2 VCOs, the Interpolating Scanner, and a few modulation sources. Sometimes it's advisable not to do too much at the same time to keep track of what you are actually doing (an oscilloscope is helpful). But gladly the "gravity" to come up with musically pleasant sounds is high enough - so just go and experiment !

Building the Interpolating Scanner

The 8 VCAs and their input attenuators are no problem. Any decent OTA chip could do the job. I have chosen the SSM 2024 from Analog Devices. It's rather inexpensive, SNR is much better than a 3080, and there are 4 OTAs in one package. If you want to use other chips like the LM3080,

LM13600 or CA3280, this should work as well. But keep in mind that the control inputs of the 2024 are referenced to (approx.) GND level, while the other chips use the negative supply as reference for an internal current mirror. You will need an extra PNP transistor with its base on GND for each control input then, or you have to change the circuit of the triangle function generators.

The triangle function generators are the tricky part. You need 8 overlapping, triangle shaped window functions for the Scan Input CV. It's important that all these functions sum up to constant gain for any CV all the way between selecting the 1st and 8th channel. My first throw was using precision rectifiers, voltage limiters and subtraction circuits based on OpAmps. But this would have lead to a very large overall circuit for 8 channels. So I built a single transistor phase splitter circuit instead, like it is used for saw to triangle conversion in many synthesizers. The idea is to have equal resistors in the emitter and collector path of a transistor. With varying input voltage the emitter will follow the base, and the collector voltage will go into the opposite direction. This is, until the two voltages meet each other as the transistor is fully turned on. Then the transistor goes into saturation mode, and the collector will follow the emitter voltage. Therefore the collector voltage (and also the collector current) performs the desired triangle function when the base and emitter are swept in one direction.

The good thing is that the collector current can directly drive the Iabc control input of an OTA. So the only additional thing for the whole circuit of one channel is a low impedance voltage source and something to set the desired offset where the individual triangle window shall begin. One opamp will fulfill both functions.

The bad thing is that we need a second PNP transistor to drive the status LED of each channel. This parallel structure is not very elegant. An optimal solution would put the LED in series with the OTA control input. The LED's drop voltage would have to be considered, but no problem here too.

Unfortunately a LED needs a few milliamperes for proper operation, while the SSM2024 needs 500uA maximum. So if I would build this module again, I'd consider using different VCAs. The CA3280 comes to mind, or a discrete solution. Thinking of it, a full OTA for each channel is overkill, anyway. One differential pair for each channel is enough, with the 16 collectors directly connected in two bus wires, and one current mirror shared by all channels.

Anyway, here is the schematics drawing of the original circuit as I built it. While it is surely not perfectly optimized, I know for certain that this circuit works, at least.

[Block Diagram](#)
[Schematics](#)

Happy building (for private use only, as usual), and please tell me about your experiences when you have built one!

For more information, please contact
[Juergen Haible](#)

All drawings copyright J. Haible (C)1996